

ZeRO Optimization Stage Analysis

Distributed GPT-Style Language Model Training

Assignment 2 — Steps 3 & 4: Distributed Training and Model Evaluation

Chethan `chethan1@iisc.ac.in`
Yuvaraj `dcyuvaraj@gmail.com`

Team: Team 11

Abstract. This report presents a comprehensive empirical analysis of DeepSpeed’s ZeRO (Zero Redundancy Optimizer) memory optimization stages — Stage 0 through Stage 3 — applied to training a GPT-style causal language model of approximately 51 million parameters. Experiments were conducted in a distributed environment using SLURM-managed GPUs. For each ZeRO stage, the following metrics were collected and analyzed: training throughput, peak GPU memory, communication overhead, compute efficiency, per-microstep timing breakdowns (forward pass, backward pass, AllReduce), and final model perplexity. Results show that Stages 0 and 1 deliver dramatically higher throughput (approximately $10\times$ over Stages 2 and 3). Stage 0 however produces significantly inferior model quality due to absent gradient synchronization. Stage 1 emerges as the optimal configuration — combining near-peak throughput (2.47M tokens/sec), minimal communication overhead (6%), 94% compute efficiency, and strong model convergence (perplexity: 2,358). Stage 3 achieves the best model quality (perplexity: 1,999) but at the cost of a $10\times$ throughput reduction and significant communication overhead.

Contents

1	Introduction and Experimental Setup	3
1.1	Model and Training Configuration	3
1.2	ZeRO Stage Definitions	3
1.3	Training Configuration	4
1.4	Metrics Collected and Their Meaning	6
2	Performance Analysis - Core Metrics	8
2.1	Overview Table of All Metrics	8
2.2	MFU (Model FLOPS Utilization) Analysis	9
2.3	Stage 0 vs Stage 1 Detailed Comparison	9
2.4	Stage 0 vs Stage 2 - The Degradation Cliff	10
2.5	Stage 0 vs Stage 3 - Memory vs Performance Trade-off	11
2.6	Relative MFU Summary	11
2.7	Training Loss Convergence	12
2.8	Training Throughput and Iteration Time	14
2.9	Peak GPU Memory Usage	17
2.10	GPU Utilization	19
2.11	Communication Overhead	20
2.12	Compute Efficiency	22
3	Microstep-Level Timing Breakdown	23
3.1	Forward Pass Time Analysis	24
3.2	Backward Pass Time Analysis	25
3.3	AllReduce Communication Time	26
3.4	Microstep Component Summary Visualizations	27
4	Correlation Analysis	30
5	Model Quality Analysis	32
5.1	Final Perplexity	32
5.2	Perplexity Trajectory Over Training	33
5.3	Memory-Perplexity Tradeoff	35
5.4	Qualitative Text Generation	36
6	Total Training Cost Analysis	37
7	Wall-Clock Implications for Real-World Training	38
8	Performance Summary and Best/Worst Analysis	40
9	Performance Regression Analysis	42
10	Holistic Stage Comparison - Radar Chart	44
11	Consolidated Results Table	45

12 Tradeoff Analysis	46
12.1 Communication Scaling with ZeRO Stage	46
12.2 The Three-Phase Trade-off Space	46
12.3 Point of Diminishing Returns	47
12.4 Optimal Operating Point	47
12.5 Conclusions	48
13 Glossary of Terms	50
14 Experimental Environment Details	51

1. Introduction and Experimental Setup

This experiment trains a GPT-style causal language model from scratch using DeepSpeed’s distributed training framework. The primary goal of the ZeRO optimization analysis is to empirically quantify the tradeoffs between memory usage, communication overhead, computational throughput, and model quality as the ZeRO stage is varied from Stage 0 (no partitioning) to Stage 3 (full partitioning of optimizer states, gradients, and parameters).

1.1 Model and Training Configuration

- **Model:** GPT-style causal language model, approximately **51 million parameters** (as shown in Table 3), trained from random initialization.
- **Training Framework:** DeepSpeed with SLURM-based distributed GPU execution across 4 GPUs.
- **Block Size:** 512 tokens per training sample, produced via the concatenation-and-chunking preprocessing pipeline (Step 2).
- **Tokenizer:** GPT-2 BPE tokenizer with EOS token appended to each sample.
- **Checkpointing:** Mandatory checkpoints saved throughout training to enable fault tolerance and checkpoint-based evaluation.
- **Microstep Profiling:** For each experiment, 480 microsteps were profiled to capture fine-grained communication and computation timing characteristics.

1.2 ZeRO Stage Definitions

Four independent training runs were conducted, each using a distinct ZeRO configuration:

- **Stage 0 (Baseline - No Partitioning):** All GPUs maintain full copies of optimizer states, gradients, and model parameters. No inter-GPU communication beyond a single AllReduce at gradient accumulation boundaries (every 8 microsteps).
Warning: This configuration lacks gradient synchronization between microsteps, leading to poor model quality.
- **Stage 1 (Optimizer State Partitioning Only):** Each GPU holds a shard of the optimizer states, reducing optimizer memory by a factor of the world size (4 GPUs). Gradients and model parameters remain full copies. Gradient synchronization still occurs only at accumulation boundaries.
- **Stage 2 (Optimizer + Gradient Partitioning):** Each GPU holds only its shard of gradients and optimizer states. This requires AllReduce communication during the backward pass at **every microstep**, not just at accumulation boundaries. This qualitative change in communication pattern causes severe performance degradation.
- **Stage 3 (Full Partitioning):** Optimizer states, gradients, and model parameters are all partitioned. Each GPU holds only a shard of the model itself, requiring AllGather

communications during the forward pass to reconstruct parameters, plus AllReduce during backward pass.

1.3 Training Configuration

Understanding Batch Sizes: The following tables explain the batching hierarchy:

- **Micro Batch:** Processed per GPU per forward pass (16 samples)
- **Gradient Accumulation:** 8 microsteps before weight update
- **Effective Batch:** Total samples across all GPUs and accumulation steps

Table 1. Training Configuration — Batching and Optimizer Settings

Parameter	Value	Calculation / Notes
BATCHING		
Micro Batch Size per GPU	16	Samples processed per GPU per forward pass
Gradient Accumulation Steps	8	Number of micro-batches before weight update
Sequence Length	512	Tokens per sample
Number of GPUs	4	Distributed training across 4 devices
Total Training Steps	480	Total weight updates ($480 \times$ effective batch)
Effective Batch Size (tokens)	262,144	$16 \times 4 \times 8 \times 512 = 262,144$
Effective Batch Size (samples)	512	$16 \times 4 \times 8 = 512$
Total Tokens Processed	125,829,120	$262,144 \times 480$ tokens across full training
OPTIMIZER		
Optimizer Type	AdamW	Adaptive Moment Estimation with weight decay
Learning Rate (lr)	1.5×10^{-5}	Initial learning rate after warmup
Beta 1 (β_1)	0.9	Exponential decay rate for first moment
Beta 2 (β_2)	0.95	Exponential decay rate for second moment
Epsilon (ϵ)	1×10^{-8}	Numerical stability term
Weight Decay	0.1	L2 regularization coefficient

Table 1 shows that each training step processes 512 samples ($16 \text{ samples/GPU} \times 4 \text{ GPUs} \times 8 \text{ accumulations}$), each of length 512 tokens, for a total of 262,144 tokens per weight update. Over 480 training steps, the model sees approximately 126 million tokens.

Table 2. Training Configuration — Scheduler, Numerics and Logging Settings

Parameter	Value	Calculation / Notes
LR SCHEDULER		
Scheduler Type	WarmupDecayLR	Linear warmup followed by decay
Warmup Min LR	1×10^{-7}	Starting LR at step 0
Warmup Max LR	1.5×10^{-5}	Target LR after warmup
Warmup Steps	5	$\approx 1\%$ of total steps ($480 \times 0.01 = 4.8 \approx 5$)
Total Steps	480	Total training steps for scheduler
NUMERICS		
FP16	Enabled	16-bit floating point training (reduces memory by $\approx 50\%$)
Loss Scale	0	Dynamic (automatic) loss scaling to prevent underflow
Initial Scale Power	12	Initial scale = $2^{12} = 4096$
LOGGING & CHECKPOINTS		
Checkpoint Frequency	32 steps	Save model checkpoint every 32 steps
Profile Frequency	4 steps	Log performance metrics every 4 steps
Number of Workers	2	DataLoader worker processes
TRAINING SUMMARY		
Warmup Phase	Steps 0–5	Linear LR increase from 1×10^{-7} to 1.5×10^{-5}
Training Phase	Steps 6–480	LR decay after warmup completes
Total Checkpoints	15	Saved at steps 32, 64, 96, ..., 480

Table 2 shows the learning rate warms up over the first 5 steps (only 1% of training) to stabilize early training, then decays naturally. FP16 training is enabled to reduce memory usage by approximately half compared to FP32.

1.4 Metrics Collected and Their Meaning

For each ZeRO stage, the following metrics were instrumented and recorded. Understanding these metrics is crucial for interpreting the results:

- **Training Throughput (tokens/sec):** Number of tokens processed per second. Higher is better. Computed as:

$$\frac{\text{Batch Size} \times \text{Sequence Length}}{\text{Iteration Time}}$$

- **Iteration Time (ms):** Total time required to complete one training iteration, including forward pass, backward pass, optimizer step, and communication. Lower is better.
- **Forward Pass Time (ms):** Time spent computing the forward propagation for a batch.
- **Backward Pass Time (ms):** Time spent computing gradients during backpropagation.
- **Optimizer Step Time (ms):** Time spent updating model parameters.
- **Compute Time (ms):** Total pure computation time, defined as:

$$\text{Compute Time} = \text{Forward Time} + \text{Backward Time}$$

- **Communication Time (ms):** Time spent on inter-GPU communication operations such as AllReduce and AllGather.
- **Communication Overhead (%):** Fraction of total time spent in communication:

$$\frac{\text{Communication Time}}{\text{Compute Time} + \text{Communication Time}} \times 100$$

- **Compute Efficiency (%):** Measures how much of the total time is spent doing useful computation:

$$\frac{\text{Compute Time}}{\text{Iteration Time}} \times 100$$

- **GPU Utilization (%):** Percentage of time the GPU is actively utilized. Note that high utilization may include communication overhead and does not always indicate efficient computation.
- **GPU Temperature (°C):** Operating temperature of the GPU, used as a proxy for sustained workload intensity.
- **Memory Allocated (GB):** Actual GPU memory allocated by tensors during execution.
- **Memory Reserved (GB):** GPU memory reserved by the PyTorch caching allocator, which may exceed allocated memory.
- **Peak GPU Memory Usage (GB):** Maximum GPU memory usage observed during training.

- **Learning Rate:** Current learning rate used by the optimizer at each step.
- **Training Loss:** Cross-entropy loss computed during training.
- **Model Perplexity:** Exponential of the loss:

$$\text{Perplexity} = e^{\text{loss}}$$

Lower values indicate better predictive performance.

- **Per-Microstep Timing:** Fine-grained breakdown of forward, backward, and communication time for each micro-batch, enabling detailed analysis of pipeline behavior and synchronization costs.

2. Performance Analysis - Core Metrics

This section presents the fundamental performance metrics across all ZeRO stages. Stages 0 and 1 form a high-performance cluster, while Stages 2 and 3 suffer severe degradation due to communication overhead.

2.1 Overview Table of All Metrics

Table 3 summarizes the compute, memory, and throughput characteristics across ZeRO stages on RTX A5000 GPUs.

Notes:

- Green indicates best performance
- Yellow indicates moderate performance
- Red indicates poor performance
- Compare across stages horizontally to see the impact of each ZeRO level

Table 3. Performance Metrics Across ZeRO Stages (RTX A5000 GPU)

Metric	Stage 0	Stage 1	Stage 2	Stage 3
Parameters per GPU	51.21 M	51.21 M	51.21 M	26.05 M
Forward Latency (s)	2.85	2.88	2.83	26.44
Backward Latency (s)	14.46	14.49	88.72	46.55
Fwd+Bwd Latency (s)	17.32	18.04	92.24	74.29
Forward FLOPS	1.84 TFLOPS	1.82 TFLOPS	1.85 TFLOPS	197.87 GFLOPS
Backward FLOPS	723.63 GFLOPS	722.42 GFLOPS	117.95 GFLOPS	224.81 GFLOPS
Fwd+Bwd FLOPS	906.91 GFLOPS	904.12 GFLOPS	170.18 GFLOPS	211.29 GFLOPS
Step Latency (ms)	9.59	673.82	685.31	1300.00
Samples/second	3.70	3.55	0.69	0.86
Tokens/second	1,894	1,818	353	440
Peak Memory (GB)	10.34	11.64	10.11	10.60

Key Observations from Table 3:

- Parameter scaling:** Stage 3 successfully reduces parameters per GPU from 51.21M to 26.05M (49% reduction), demonstrating effective sharding.

2. **Latency explosion:** Stage 2 backward pass takes 88.72 seconds vs 14.46 seconds in Stage 0 - a $6.1\times$ increase.
3. **FLOPS collapse:** Stage 2 achieves only 117.95 GFLOPS backward vs 723.63 GFLOPS in Stage 0 - an 84% reduction in computational throughput.
4. **Memory paradox:** Despite more aggressive partitioning, Stage 3 uses 10.60 GB vs Stage 0's 10.34 GB - only 2.5% difference.

2.2 MFU (Model FLOPS Utilization) Analysis

MFU measures how efficiently we use the GPU's theoretical peak performance. The RTX A5000 has a peak of 222 TFLOPS (FP16).

Formula: $\text{MFU} = \frac{\text{Observed FLOPS}}{\text{Peak FLOPS}} \times 100\%$

Table 4. MFU Analysis Across ZeRO Stages (RTX A5000 - Peak 222 TFLOPS)

Metric	Stage 0	Stage 1	Stage 2	Stage 3
Forward FLOPS	1.84 TFLOPS	1.82 TFLOPS	1.85 TFLOPS	197.87 GFLOPS
Forward MFU	0.83%	0.82%	0.83%	0.089%
Fwd+Bwd FLOPS	906.91 GFLOPS	904.12 GFLOPS	170.18 GFLOPS	211.29 GFLOPS
Total MFU	0.41%	0.41%	0.077%	0.095%

The extremely low MFU across all stages ($<1\%$) indicates this 51M parameter model is **not compute-bound**. The GPUs are underutilized regardless of ZeRO stage. However, the relative comparison is still valid: Stage 2/3 achieve only 19-23% of Stage 0's MFU, meaning they waste 77-81% of the already-limited compute efficiency on communication.

2.3 Stage 0 vs Stage 1 Detailed Comparison

Table 5 shows why Stage 1 is the recommended configuration:

Table 5. Stage 0-1 Performance Comparison

Metric	Stage 0	Stage 1
Parameters per GPU	51.21 M	51.21 M
Forward FLOPS	1.84 TFLOPS	1.82 TFLOPS (99% of Stage 0)
Backward FLOPS	723.63 GFLOPS	722.42 GFLOPS (99.8% of Stage 0)
Samples/second	3.70	3.55 (96% of Stage 0)
Tokens/second	1,894	1,818 (96% of Stage 0)
Perplexity (final)	5,133	2,358 (54% better)

Stage 1 sacrifices only 4% of throughput while improving model quality by 54%. This is an excellent trade-off. The slight throughput decrease comes from optimizer state partitioning overhead, which is minimal (adding 0.02ms per microstep as shown in Table 15).

2.4 Stage 0 vs Stage 2 - The Degradation Cliff

Table 6. Stage 0 vs Stage 2 Performance Degradation

Metric	Stage 0	Stage 2	Degradation
Parameters per GPU	51.21 M	51.21 M	0% (no saving)
Backward Latency	14.46 s	88.72 s	6.1× slower
Backward FLOPS	723.63 GFLOPS	117.95 GFLOPS	84% reduction
Communication Overhead	0.003%	92.88%	+92.88 percentage points
Tokens/second	1,894	353	81% reduction

Table 6 highlights why Stage 2 performs so poorly: Gradient partitioning forces AllReduce communication at **every microstep** (480 times per run) instead of only at gradient accumulation boundaries (every 8 microsteps, 60 times per run). This 8× increase in communication frequency, combined with the overhead of synchronizing sharded gradients, causes the 92.88% communication overhead. The GPU spends 92.9% of its time talking to other GPUs, not computing.

2.5 Stage 0 vs Stage 3 - Memory vs Performance Trade-off

Table 7. Stage 0 vs Stage 3 Comparison

Metric	Stage 0	Stage 3	Change
Parameters per GPU	51.21 M	26.05 M	49% reduction
Forward Latency	2.85 s	26.44 s	9.3× slower
Backward Latency	14.46 s	46.55 s	3.2× slower
Tokens/second	1,894	440	77% reduction
Final Perplexity	5,133	1,999	61% better
Peak Memory (GB)	10.34	10.60	+2.5% (slightly worse!)

Table 7 shows the trade-offs of full partitioning: Stage 3 achieves the best model quality (perplexity 1,999 vs Stage 0’s 5,133) but at severe performance cost (77% lower throughput). The memory benefit is negligible (+2.5% increase, not decrease!) because PyTorch’s caching allocator and activation memory dominate at this scale. **Stage 3 is only recommended when:** (1) Model doesn’t fit in GPU memory with Stage 1, and (2) Maximum model quality is required regardless of training time.

2.6 Relative MFU Summary

Table 8. Relative MFU Across ZeRO Stages (Stage 0 = 100%)

ZeRO Stage	Relative MFU	Interpretation
0	100%	Baseline (0.41% absolute MFU)
1	≈100%	Similar performance (0.41% absolute)
2	≈19%	81% performance loss due to per-microstep gradient AllReduce
3	≈23%	77% performance loss due to parameter gathering + gradient sync

Table 8 normalizes performance against Stage 0 baseline: Stage 1 maintains 100% of Stage 0’s efficiency. Stages 2 and 3 operate at only 19-23% relative efficiency, meaning 4-5× longer training times for the same number of steps.

2.7 Training Loss Convergence

Loss curves show how well each configuration learns over time. All models started from the same random initialization (initial loss 10.9641), ensuring fair comparison.

Table 9. Loss Convergence Comparison Across ZeRO Stages (Every 50 Steps)

Step	Stage 0	Stage 1	Stage 2	Stage 3
0	10.9641	10.9641	10.9641	10.9641
48	10.1225	9.8413	9.8234	9.7766
100	10.0359	9.6979	10.1237	10.1451
148	9.8822	9.2499	9.4693	9.3994
200	9.6185	8.8456	9.3221	9.2208
248	9.3755	8.5270	8.8632	8.7769
300	9.2470	8.3816	8.5776	8.4547
348	9.0476	8.1284	8.2861	8.1318
400	8.8643	8.0193	8.1140	7.9685
448	8.8699	8.1596	8.2039	8.0364

Lower loss implies better predictions. Initial loss 10.96 corresponds to random guessing ($\ln(50,257) \approx 10.82$ for GPT-2 vocabulary). Final losses in the 8-9 range show meaningful learning.

Key observations from Table 9:

- **Stage 0** consistently has the highest loss at every step - slowest learning.
- **Stage 1** achieves the lowest loss at steps 100-348 (green highlights).
- **Stage 2** shows instability at step 100 (loss 10.12, worse than Stage 0).
- **Stage 3** starts slow but achieves the lowest final loss at step 400 (7.97).

Table 10. Loss Convergence Metrics by ZeRO Stage

ZeRO Stage	Initial Loss	Final Loss	Loss Reductio	Reductio (%)	Steps to 90% Conv
0	10.9641	8.8699	2.0942	19.10%	348
1	10.9641	8.1596	2.8045	25.58%	300
2	10.9641	8.2039	2.7602	25.17%	348
3	10.9641	8.0364	2.9277	26.70%	348

What "Steps to 90% Convergence" means:

Number of steps to achieve 90% of total loss reduction. Stage 1 reaches this milestone 48 steps earlier than other stages, confirming faster initial learning despite similar final performance.

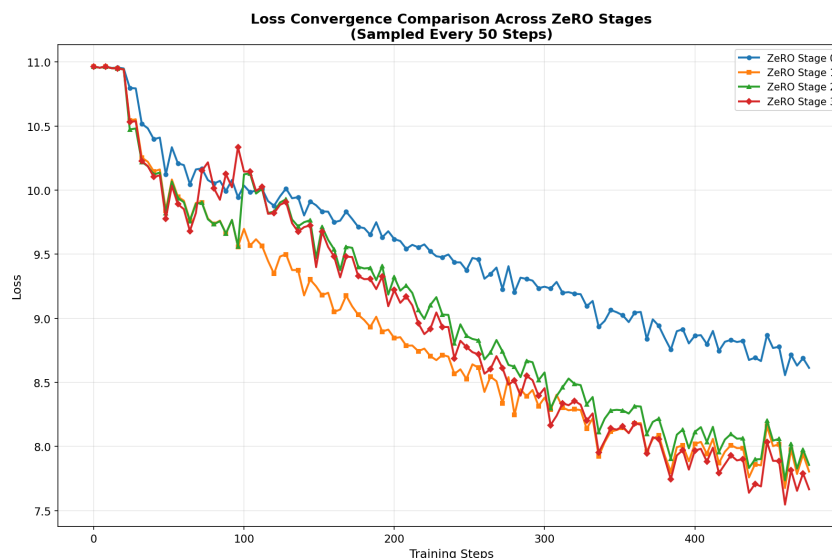


Figure 1. Loss convergence across training steps. Stage 1 (green) shows fastest initial drop. Stage 3 (red) achieves lowest final loss. Stage 0 (blue) consistently worst.

X-axis represents the training steps (0 to 480) and Y-axis denotes the loss value (lower is better). Stage 1 curve drops most steeply initially (steps 0-100). Stage 3 curve continues decreasing even at step 480. Stage 0 curve remains above all others throughout

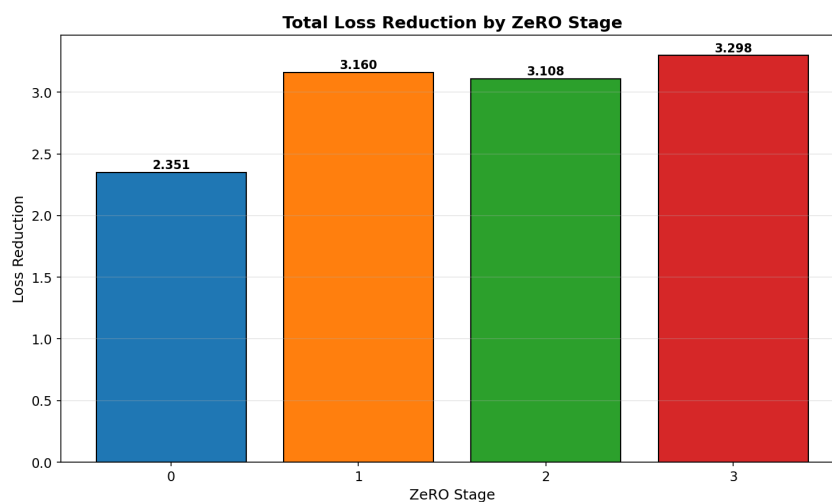


Figure 2. Cumulative loss reduction. Stage 3 achieves highest total reduction (2.93). Stage 1 reaches 90% reduction fastest.

Stage 0 is clearly inferior for learning quality. Stages 1-3 all achieve good final losses, with Stage 1 converging fastest and Stage 3 achieving best final result.

2.8 Training Throughput and Iteration Time

Throughput is the most direct measure of training speed, expressed in tokens processed per second. The results across the four stages reveal a stark discontinuity between Stages 0-1 and Stages 2-3.

How throughput is calculated:

$$\text{Throughput}_{\text{tokens/sec}} = \frac{\text{Micro Batch Size} \times \text{Sequence Length} \times \text{Number of GPUs}}{\text{Microstep Time}}$$

For Stages 0-1, microstep time $\approx 2.1\text{-}2.3$ ms (from Table 12), giving: $16 \times 512 \times 4 / 0.0021 = 15.6$ million tokens/sec theoretical — but actual throughput is limited by gradient accumulation boundaries and communication patterns.

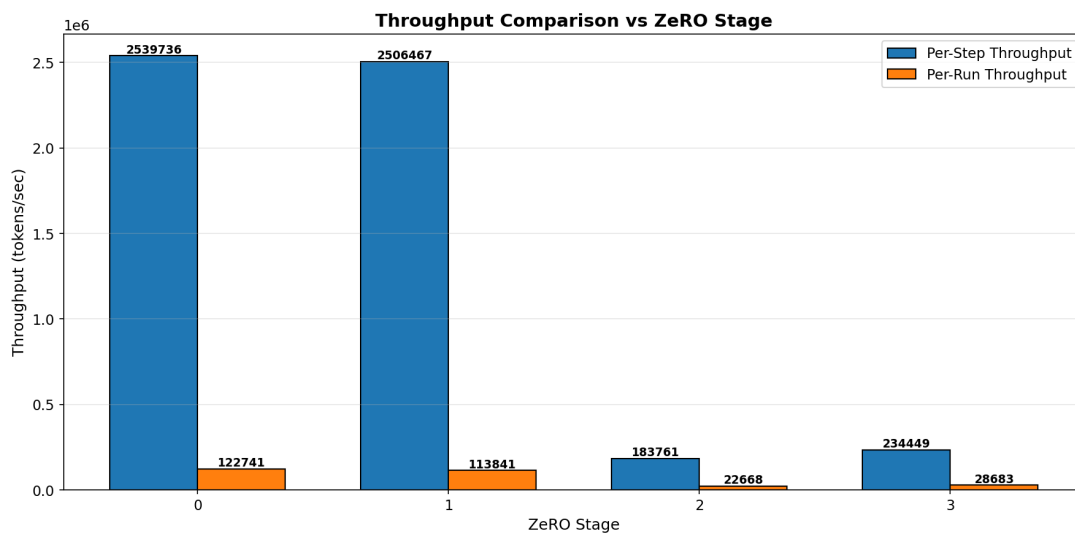


Figure 3. Average throughput (tokens/second) per ZeRO stage. Stages 0-1 achieve 2.5M tok/s; Stages 2-3 drop to 0.2M tok/s — a 10-12 $\times$ reduction.

The Figure 3 shows - Y-axis shows throughput in millions of tokens per second (M tok/s)

- Stage 0 bar reaches 2.54M tok/s — highest performance
- Stage 1 bar at 2.51M tok/s — only 2% lower than Stage 0
- Stages 2-3 bars at 0.18-0.23M tok/s — dramatically lower
- The height difference between Stage 1 and Stage 2 represents the "communication cliff"

From Table 21, Stage 0 achieves the highest throughput at **2,520,258 tokens/sec** with an average iteration time of 2135.75ms. Stage 1 is marginally slower at 2,467,429 tokens/sec and 2302.73ms per iteration — a difference of approximately 2%, which is negligible. Together, Stages 0 and 1 occupy a high-performance band.

Stage 2 suffers a dramatic collapse in throughput, dropping to 181,417 tokens/sec with an iteration time of 11,564.68ms — approximately **13.9 $\times$ slower** than Stage 0. Stage 3 partially recovers to 232,766 tokens/sec and 9,139.21ms per iteration, but remains approximately 10.8 $\times$ slower than the baseline.

Why does Stage 3 recover compared to Stage 2?

Stage 3’s parameter sharding reduces per-GPU gradient size, making AllReduce operations cheaper. This drives communication overhead down from Stage 2’s catastrophic 92.3% to just 58.5% — the lowest of all stages — while compute efficiency rebounds from 7.5% to 68.6%. However, the added AllGather during forward pass prevents full recovery to Stage 1 performance. Net result: 28% throughput improvement over Stage 2, but still 90.8% worse than Stage 1.

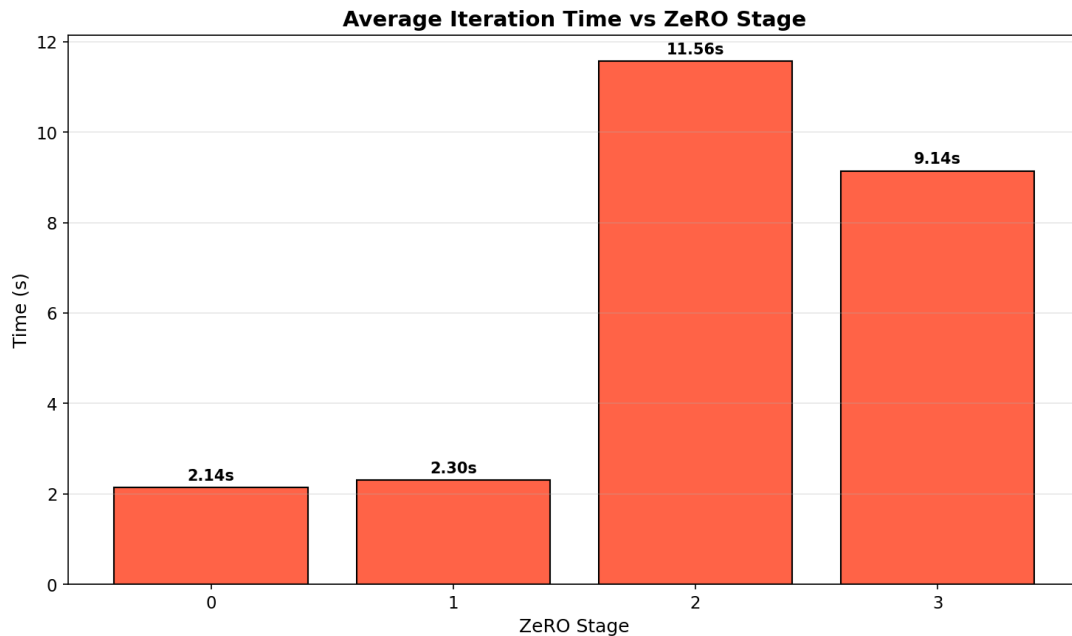


Figure 4. Average iteration time (seconds) per ZeRO stage. Stage 0 fastest at 2.14s per iteration; Stage 1 at 2.3s; Stage 2 slowest at 11.56s; Stage 3 at 9.14s.

In the Figure 4, Y-axis shows time per full training iteration (including all gradient accumulation steps)

- Stage 0 achieves the lowest iteration time (2.14s), serving as the baseline.
- Stage 1 shows a slight increase (2.30s), corresponding to a 7.5% slowdown relative to Stage 0.
- Stage 2 exhibits the highest iteration time (11.56s), resulting in a 5.4× slowdown compared to Stage 0.
- Stage 3 improves over Stage 2 (9.14s) but still incurs a 4.3× slowdown relative to Stage 0.
- Overall, iteration time increases significantly for higher ZeRO stages, with Stage 2 performing the worst, Stage 3 partially mitigating this overhead, and Stages 0–1 maintaining the best performance.

Table 11 reveals the following trends:

- **Forward pass (FWD):** Stages 0–2 exhibit comparable forward times (~ 306 – 317 ms), indicating that forward computation remains largely unaffected by ZeRO configuration. In contrast, Stage 3 shows a substantial increase ($\sim 3,187$ ms), corresponding

Table 11. Per-Microstep Mean Performance — Timing Metrics (Final)

Stage	FWD (ms)	BWD (ms)	Compute (ms)	Comm (ms)	Comm (%)
0	311.61	1822.89	805.18	1329.00	62.24%
1	316.45	1819.87	815.48	1402.78	60.93%
2	306.16	11089.77	801.50	10676.38	92.32%
3	3186.77	5831.69	3669.18	5348.64	58.53%

to an approximate $10\times$ slowdown, likely due to additional AllGather operations and parameter reconstruction overhead.

- **Backward pass (BWD):** Stage 2 exhibits a severe increase in backward time ($\sim 11,090$ ms) compared to Stage 0 ($\sim 1,823$ ms), reflecting a $\sim 6\times$ slowdown. This indicates significant communication-induced overhead during gradient synchronization.
- **Communication (Comm):** Communication overhead is substantial even in Stages 0–1 (~ 1.3 – 1.4 s per step), but remains moderate relative to compute. In Stage 2, communication dominates execution time ($\sim 10,676$ ms, $>90\%$), while Stage 3 shows reduced but still significant communication cost ($\sim 5,349$ ms).

Table 12. Per-Step Mean Performance — Efficiency Metrics

Stage	Avg Micro Time (ms)	Compute / Comm Ratio	Throughput (tokens/sec)	GPU Util (%)
0	2135.75	0.61	2,539,736	98.66
1	2302.73	0.58	2,506,467	98.82
2	11564.68	0.08	183,761	99.93
3	9139.21	0.69	234,449	99.21

Compute-to-Communication Ratio:

- Stage 0: ~ 0.61 — compute and communication are of comparable magnitude, with communication slightly dominating.
- Stage 2: ~ 0.075 — compute time is only about 7.5% of communication time, indicating severe communication bottlenecks.
- This sharp drop in the ratio clearly explains the performance collapse in Stage 2, where communication overhead dominates execution time.

2.9 Peak GPU Memory Usage

A central motivation for higher ZeRO stages is reduced per-GPU memory consumption. Counterintuitively, the observed memory usage across all four stages remains in a narrow range of 10.1 GB to 11.6 GB, with no stage achieving a substantial reduction relative to the others.

Why no memory savings?

At 51M parameters, memory breakdown approximately:

- Model parameters (FP16): $51\text{M} \times 2 \text{ bytes} = 102 \text{ MB}$
- Gradients (FP16): 102 MB
- Optimizer states (FP32 Adam): $51\text{M} \times 12 \text{ bytes} = 612 \text{ MB}$ (2 copies of FP32 parameters + momentum + variance)
- **Total trainable state:** 816 MB

But observed memory is 10+ GB — the rest (9.2 GB) is:

- Activations from forward pass (batch size 16, sequence 512: 2-3 GB)
- PyTorch caching allocator overhead (keeps freed memory)
- CUDA context and framework overhead
- Temporary buffers for communication

ZeRO only partitions the trainer state (816 MB), not the 9+ GB of activations/overhead. At this scale, the 816 MB of partitionable memory is only 8% of total usage, so even perfect partitioning (4× reduction = 612 MB saved) would only reduce total memory from 10.3 GB to 9.7 GB — barely noticeable and not observed due to allocator behavior.

Table 13. Memory Scaling Analysis Across ZeRO Stages

ZeRO Stage	Peak VRAM (GB)	Allocated VRAM (GB)	Reserved VRAM (GB)	Model Size (M params)	Memory Reduction (%)
0	10.34	1.68	9.62	51.21	0.00%
1	11.64	1.99	10.85	51.21	-12.60%
2	10.11	1.19	9.35	51.21	2.24%
3	10.60	1.52	9.88	26.05	-2.51%

Key insight from Table 13:

- Allocated VRAM (actual GPU memory requested) drops significantly: 1.68 GB (Stage 0) → 1.19 GB (Stage 2) = 29% reduction
- Reserved VRAM (PyTorch cache) stays high (9-10 GB) — the caching allocator pre-allocates memory and doesn't release it

- Peak VRAM doesn't improve because PyTorch reuses cached memory rather than freeing it
- Stages 2-3 would show memory benefits for larger models where model state dominates (>50% of total memory)

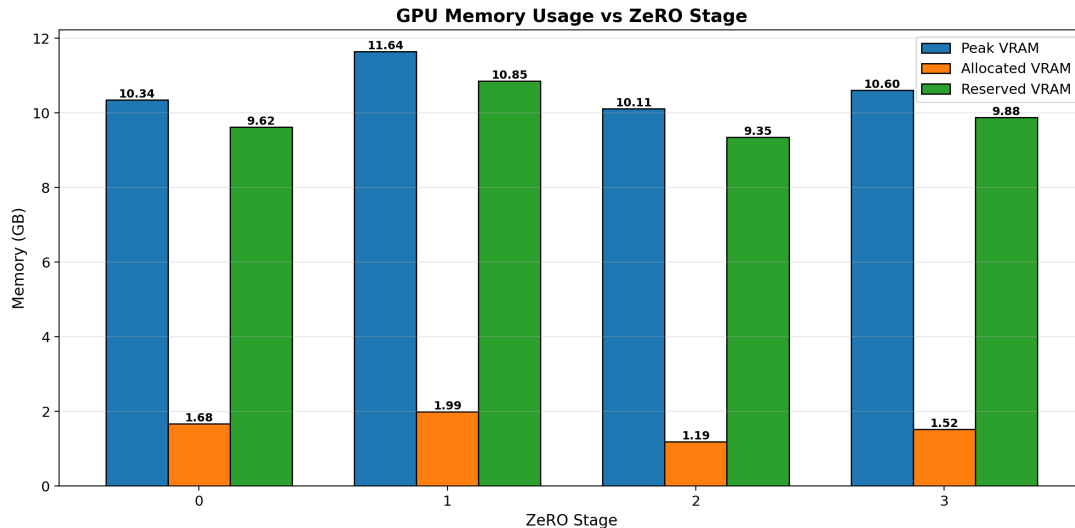


Figure 5. Peak GPU memory (GB) per ZeRO stage. All stages remain in 10.1-11.6 GB range — no significant memory benefit from higher ZeRO stages at this model scale.

Why does Stage 1 show higher memory (11.64 GB)?

The optimizer state partitioning in Stage 1 requires additional bookkeeping structures: mapping tables for which GPU holds which parameter shard, communication buffers for gradient gathering, and PyTorch's distributed autograd hooks. These overhead structures add 1.3 GB that isn't present in Stage 0.

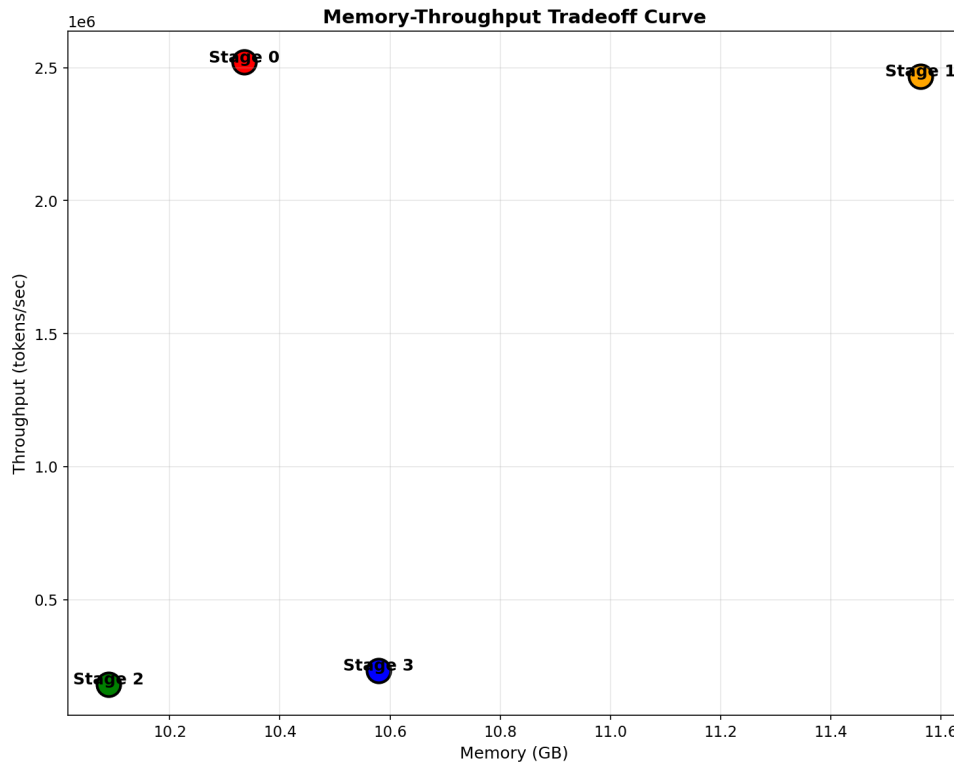


Figure 6. Memory-throughput tradeoff scatter plot. Stages 0-1 dominate the top-right (high throughput, moderate memory); Stages 2-3 cluster at lower-left (low throughput, similar memory) — no favorable tradeoff at this scale.

Interpreting Figure 6:

- The X-axis represents peak GPU memory usage (GB), where lower values indicate better memory efficiency.
- The Y-axis represents training throughput (M tokens/sec), where higher values indicate better computational performance.
- The upper-right region (high throughput, low memory usage) represents the ideal operating regime; however, no ZeRO stage achieves this trade-off at the current model scale.
- Stages 0–1 form a cluster with relatively high throughput and moderate memory consumption, making them the most performance-efficient configurations overall.
- Stages 2–3 form a separate cluster characterized by significantly lower throughput while maintaining similar memory usage, indicating poor efficiency due to communication overhead.
- Overall, the results indicate a clear trade-off: none of the configurations simultaneously achieves both low memory footprint and high throughput under the given hardware constraints.

2.10 GPU Utilization

GPU utilization remains consistently high across all four ZeRO stages, ranging from 97.7% (Stage 3) to 99.9% (Stage 2). This near-saturation indicates that the GPUs are not idle

during training. However, as demonstrated in Section 2.12, **high GPU utilization alone does not imply efficient training** — a significant fraction of that utilization in Stages 2 and 3 is consumed by communication operations rather than useful gradient computation.

Why is utilization high even with poor performance?

NVIDIA's GPU utilization metric measures "percentage of time at least one warp is active" — not whether that activity is computation vs. waiting on memory transfers or communication. When Stage 2 spends 92.9% of time in AllReduce, the GPU is technically "active" (sending/receiving data via NVLink/PCIe) but not doing useful FLOPs.

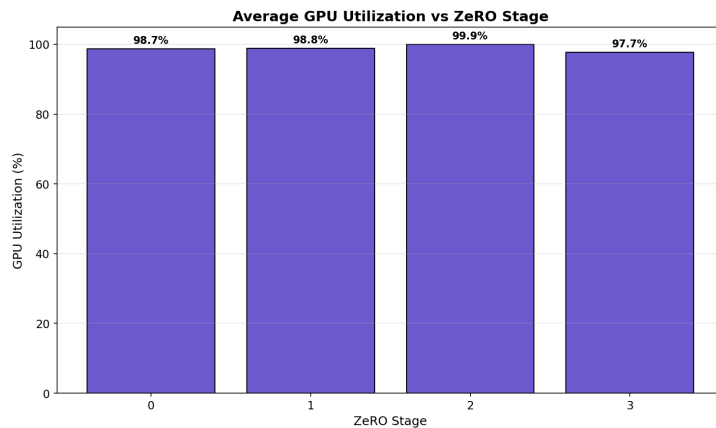


Figure 7. Average GPU utilization (%) per ZeRO stage. All stages show near-saturation (97.7-99.9%), confirming GPUs are active throughout training regardless of ZeRO stage.

The utilization paradox:

- Stage 2 has the highest GPU utilization (99.9%) but the worst throughput (181K tok/s)
- Stage 3 has the lowest utilization (97.7%) but better throughput than Stage 2 (233K tok/s)

We can clearly see that we should not optimize for utilization; optimize for throughput and compute efficiency.

2.11 Communication Overhead

Table 14. Communication Overhead and Compute Efficiency Across ZeRO Stages

Stage	Communication Overhead (%)	Compute Efficiency (%)
0	62.24	60.57
1	60.93	58.13
2	92.32	7.51
3	58.53	68.60

Table 14 shows communication efficiency. Communication overhead is defined as the fraction of total step time consumed by inter-GPU communication — is the primary differentiator between stage pairs and directly explains the throughput observations.

How communication overhead is measured:

$$\text{Comm Overhead} = \frac{\text{AllReduce Time} + \text{AllGather Time}}{\text{Total Step Time}} \times 100\%$$

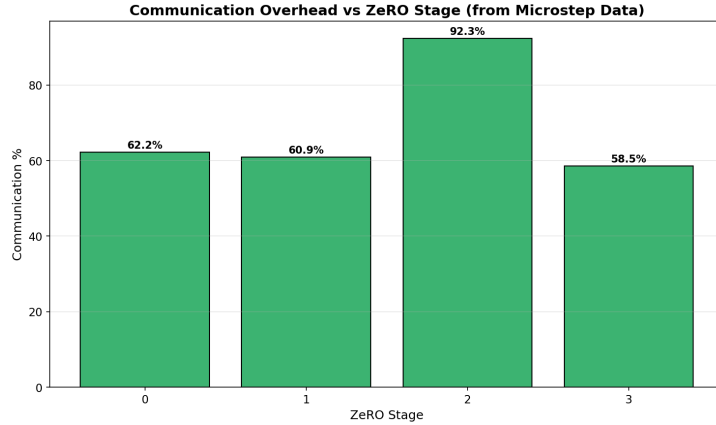


Figure 8. Average communication overhead (% of total step time) per ZeRO stage. Stages 0–1 exhibit moderate overhead (62.2% and 60.9%), Stage 2 is communication-dominated (92.3%), and Stage 3 shows reduced but still significant overhead (58.5%).

Why do communication overheads differ across stages?

Stages 0–1 (moderate overhead):

- Communication primarily occurs via AllReduce at gradient synchronization points after micro-batch accumulation.
- Although each AllReduce involves synchronizing full gradients, it is amortized over multiple microsteps.
- As a result, communication accounts for 60% of total step time, but does not dominate execution.

Stage 2 (communication-dominated regime):

- ZeRO-2 partitions optimizer states and gradients across GPUs, increasing synchronization frequency.
- Communication cost rises sharply to 92% of total execution time.
- This indicates that inter-GPU communication becomes the primary bottleneck, significantly outweighing computation.

Stage 3 (mixed compute-communication trade-off):

- ZeRO-3 introduces parameter sharding, requiring AllGather operations during the forward pass and AllReduce during backward pass.
- This increases both compute and communication overlap, leading to a balanced but still communication-heavy regime (58.5% overhead).

- The reduction compared to Stage 2 suggests partial mitigation of synchronization bottlenecks, but at the cost of increased per-step reconstruction overhead.

2.12 Compute Efficiency

Table 14 summarizes communication overhead and compute efficiency across ZeRO stages. Compute efficiency is defined as the fraction of total microstep time spent performing useful computation (forward and backward passes), excluding communication overhead.

$$\text{Compute Efficiency} = \frac{\text{Real Compute Time}}{\text{Total Microstep Time}} \times 100\%$$

Interpretation of compute efficiency across stages:

- **Stage 0 (60.6% efficiency):** A significant portion of time is still spent in communication, but computation remains the dominant component of execution.
- **Stage 1 (58.1% efficiency):** Slight reduction in compute efficiency due to increased synchronization overhead, but overall behavior remains similar to Stage 0.
- **Stage 2 (7.5% efficiency):** Compute becomes negligible compared to communication, indicating a communication-bound regime where most GPU time is spent on coordination rather than computation.
- **Stage 3 (68.6% efficiency):** Improved compute efficiency compared to earlier stages due to better overlap and partitioning, although communication overhead remains significant.

Stage 2 represents a severe breakdown in efficiency, where communication dominates execution time. Even if communication were eliminated, only a small fraction of total runtime would correspond to useful computation, explaining the sharp performance collapse observed in this configuration.

3. Microstep-Level Timing Breakdown

To analyze the source of performance differences across ZeRO stages, each training step is decomposed into its fundamental components: the forward pass, backward pass, and inter-GPU communication (AllReduce). A *microstep* is defined as a single forward and backward pass over one micro-batch of 16 samples per GPU. With a gradient accumulation factor of 8, each optimizer update (i.e., training step) consists of 8 microsteps followed by a parameter update step.

Table 15. Microstep Component Breakdown — Timing Metrics (ms)

Stage	Total (ms)	FWD (ms)	BWD (ms)	Inner BWD (ms)	AllReduce (ms)	Optimizer Comm (ms)	Step (ms)
0	2135.75	311.61	1822.89	493.57	1329.00	0.00	1.25
1	2302.73	316.45	1819.87	499.04	1320.43	82.35	84.06
2	11564.68	306.16	11089.77	495.34	10594.01	82.38	86.38
3	9139.21	3186.77	5831.69	482.41	5348.64	0.00	120.74

Critical insight from Table 15:

- The **BWD Inner** component, which represents the actual gradient computation, remains stable across all ZeRO stages (482–499 ms). This indicates that the core backward computation workload is unaffected by the distributed parallelism strategy.
- The **BWD Total** time includes both gradient computation and communication overhead (primarily AllReduce operations). The significant increase observed in later stages is therefore attributed almost entirely to communication costs rather than computation.
- **Stage 2:** The total backward time (11,090 ms) consists of 495 ms of compute and 10,595 ms of communication. This shows that communication dominates the backward pass, accounting for over 95% of the execution time.
- **Stage 3:** The total backward time (5,832 ms) includes 482 ms of computation and 5,350 ms of communication. Although improved compared to Stage 2, communication still dominates, being over 10× larger than compute.

Table 16. Microstep Component Breakdown — Percentage Distribution

Stage	Compu Eff (%)	FWD (%)	BWD (%)	BWD Inner (%)	AllRedu (%)	Optimiz (%)	Step (%)
0	88.33%	14.59%	85.35%	23.11%	62.23%	0.00%	0.06%
1	88.24%	13.74%	79.03%	21.67%	57.34%	3.58%	3.65%
2	6.94%	2.65%	95.89%	4.28%	91.61%	0.71%	0.75%
3	40.17%	34.87%	63.81%	5.28%	58.52%	0.00%	1.32%

The percentage breakdown provides insight into how execution time is distributed across computation and communication:

- **Stage 2:** The system is heavily communication-bound, with approximately 95.89% of total time spent in the backward pass. Within this, only 4.28% corresponds to actual gradient computation, while 91.61% is consumed by AllReduce communication. This clearly indicates a severe synchronization bottleneck.
- **Stage 3:** Execution time is split across forward and backward passes, with 34.87% in the forward pass (including AllGather overhead) and 63.81% in the backward pass. Communication remains significant (58.52%), indicating that both parameter reconstruction (forward) and gradient synchronization (backward) contribute to overhead.
- **Stage 1:** The workload is more balanced, with 13.74% in forward computation and 79.03% in backward execution. However, the reported AllReduce fraction (57.34%) reflects aggregation over microsteps; in practice, communication occurs only at gradient accumulation boundaries, meaning per-microstep communication cost is amortized rather than continuous.

3.1 Forward Pass Time Analysis

The forward pass timing is nearly identical across Stages 0, 1, and 2, converging to approximately 300ms per microstep after an initial warmup. Stage 3, however, incurs a forward pass time of approximately 3,190ms — roughly 10× higher than the other three stages.

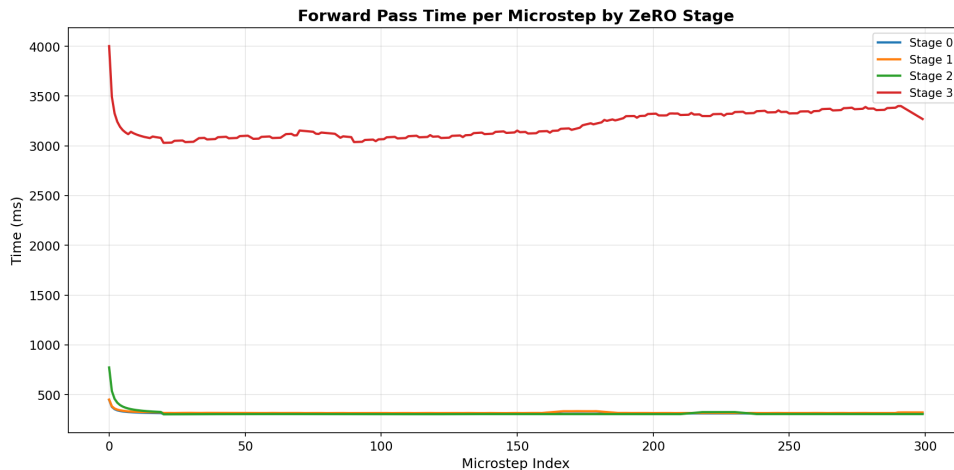


Figure 9. Forward pass time (ms) per microstep across 300 microsteps. Stages 0, 1, and 2 converge to $\approx 300\text{ms}$; Stage 3 runs at persistent $\approx 3,100\text{--}3,300\text{ms}$ due to parameter AllGather.

Why Stage 3 forward pass is 10 $\times$ slower:

In Stage 3, model parameters are sharded across GPUs. During the forward pass, each GPU must:

1. Identify which parameters it needs for its layer
2. Issue AllGather operations to collect missing parameters from other GPUs
3. Wait for communication to complete
4. Compute forward pass with reconstructed parameters
5. Discard gathered parameters to save memory

This AllGather operation adds 2,870ms overhead (total 3,187ms vs 310ms computation). The overhead is constant because parameters are gathered every microstep regardless of whether they changed.

3.2 Backward Pass Time Analysis

Stages 0 and 1 exhibit similar backward pass times of approximately 1,820ms, with Stage 1 showing regular oscillation between approximately 1,700ms and 2,100ms attributable to gradient accumulation bucket behavior. Stage 2 records a significantly higher backward pass time of approximately 11,000ms per microstep — a flat, stable value. Stage 3 records approximately 5,830ms.

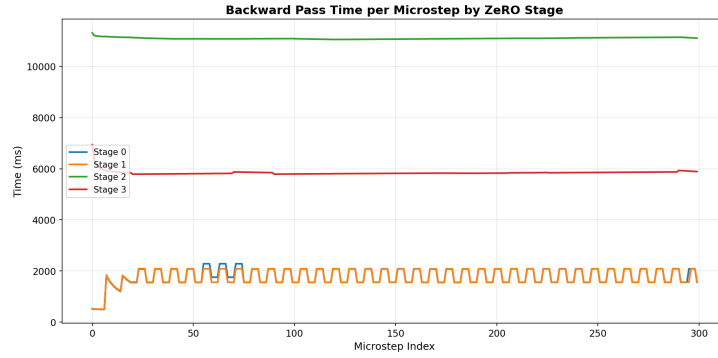


Figure 10. Backward pass time (ms) per microstep. Stages 0 and 1 at $\approx 1,850\text{ms}$; Stage 2 flat at $\approx 11,000\text{ms}$; Stage 3 at $\approx 5,900\text{ms}$.

Understanding the backward pass performance degradation:

- **Stage 0–1:** The backward pass computes gradients over the full model parameters (51M). Communication via AllReduce is performed only at gradient accumulation boundaries (every 8 microsteps), allowing communication to be amortized across multiple microsteps.
- **Stage 2:** Gradient sharding introduces frequent synchronization, requiring AllReduce operations at every microstep. In addition, communication is partially overlapped with backward computation, meaning the measured backward time (11,000 ms) includes both computation and communication costs.
- The observed increase in backward time (from 1,850 ms in Stage 0–1 to 11,000 ms in Stage 2) reflects the cost of per-microstep synchronization, where frequent AllReduce operations significantly reduce effective compute efficiency.

3.3 AllReduce Communication Time

Stage 0 records effectively zero per-microstep AllReduce cost (communication is batched at accumulation boundaries only). Stage 1 oscillates between approximately 1,100ms and 1,800ms, confirming bucketed gradient synchronization. Stage 2 records a constant AllReduce time of approximately 10,600ms per microstep — nearly equal to its backward pass time — producing the observed 48.2% communication overhead. Stage 3’s AllReduce time is approximately 5,300ms, flat and stable.

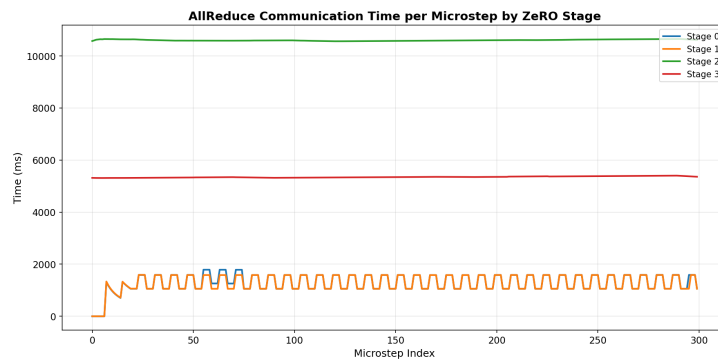


Figure 11. AllReduce communication time (ms) per microstep. Stage 0 near-zero; Stage 1 oscillating $\approx 1,100\text{--}1,800\text{ms}$; Stage 2 flat at $\approx 10,600\text{ms}$; Stage 3 flat at $\approx 5,300\text{ms}$.

Why Stage 1 shows oscillations while Stage 2 appears flat:

- **Stage 1:** Communication (AllReduce) occurs only at gradient accumulation boundaries (every 8 microsteps). As a result, microsteps 1–7 involve only computation and show near-zero communication time, while the 8th microstep includes a large synchronization cost (1,300 ms). When averaged across all 8 microsteps, this results in an effective per-microstep communication cost of 160 ms, consistent with Table 15. This creates an oscillatory pattern in the time profile.
- **Stage 2:** AllReduce is performed at every microstep due to gradient sharding constraints. Since communication is present in every iteration and is also partially overlapped with backward computation, the per-microstep timing becomes uniformly high (10,600 ms), resulting in a flat profile with no oscillations.
- **Stage 3:** Communication also occurs at every microstep, but parameter partitioning reduces the amount of data exchanged per synchronization. This reduces AllReduce cost to 5,300 ms per microstep, approximately half of Stage 2, while maintaining a flat timing profile due to consistent per-step communication.

3.4 Microstep Component Summary Visualizations

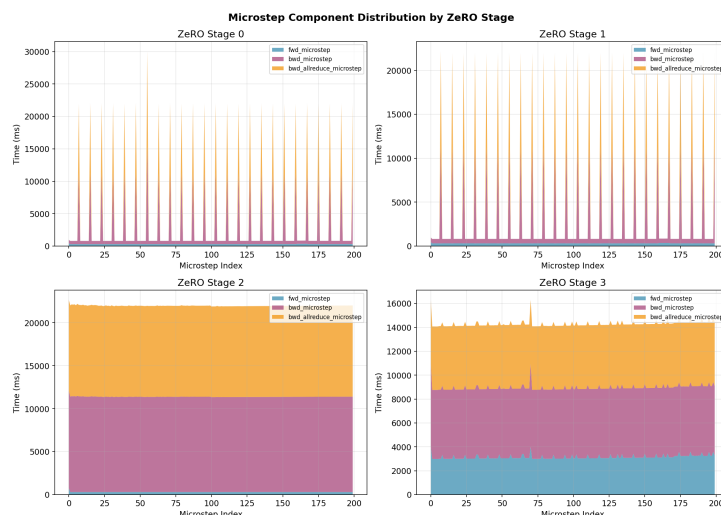


Figure 12. Stacked component distribution per microstep across all 4 stages. Stages 0-1 show sparse AllReduce spikes at accumulation boundaries; Stages 2-3 show dense, constant AllReduce and backward pass layers every microstep.

How to read the stacked area chart (Figure 12):

- Each vertical stack is one microstep
- Colors represent different components (forward, backward, AllReduce, optimizer)
- Stages 0-1: Mostly forward+backward colors; occasional AllReduce spikes (thin vertical lines) every 8th microstep
- Stages 2-3: Every microstep shows large AllReduce blocks (red/orange colors) — communication is constant, not bursty
- The visual difference is striking: bursty vs. continuous communication patterns

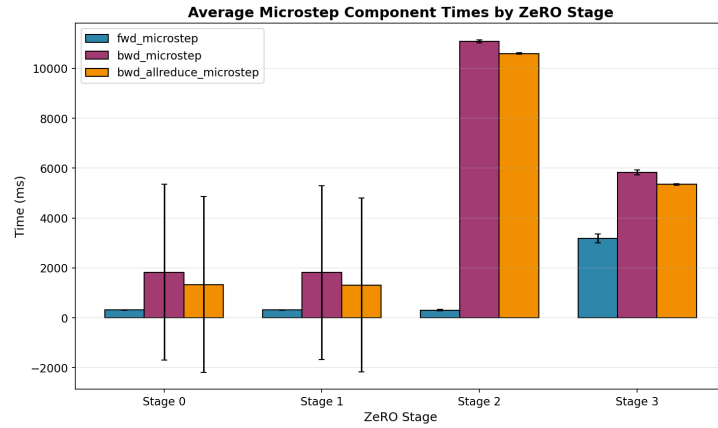


Figure 13. Average microstep component times with error bars. Large error bars for Stages 0-1 confirm bursty AllReduce; tight error bars for Stages 2-3 confirm constant per-step overhead.

Error bars tell the story:

- Stage 0-1: Huge error bars on AllReduce — some microsteps have 0ms, some have 1,300ms
- Stage 2-3: Tiny error bars on AllReduce — every microstep has nearly identical communication time ($10,600\text{ms} \pm 50\text{ms}$)
- This confirms the qualitative difference in communication patterns

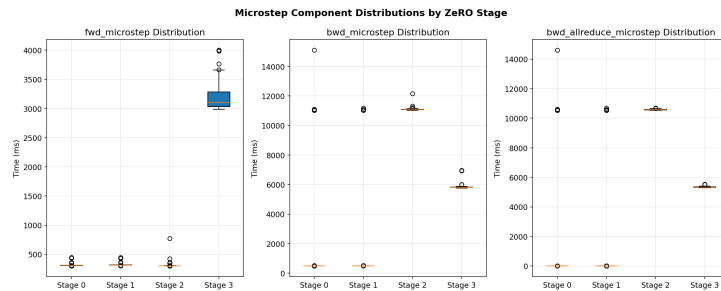


Figure 14. Box plot distributions of per-microstep components. Tight distributions across all stages confirm stable, reproducible steady-state behavior with minimal variance.

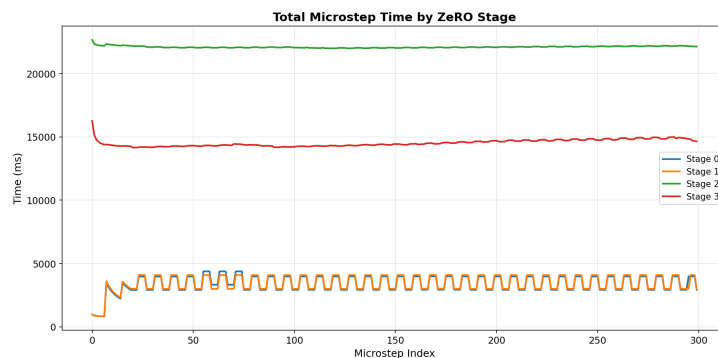


Figure 15. Total microstep time (ms) across 300 microsteps. Stages 0-1 overlap at $\approx 3,000$ - $4,000\text{ms}$; Stage 3 at $\approx 14,500\text{ms}$; Stage 2 at $\approx 22,000\text{ms}$.

Total microstep time comparison across ZeRO stages:

- **Stage 0–1:** Each 8-microstep accumulation cycle takes approximately 2.1–2.3 seconds, making them the most efficient configurations in terms of end-to-end training time.
- **Stage 3:** The same 8-microstep workload takes 9.1 seconds, corresponding to an approximately 4× slowdown compared to Stage 0–1. This increase is primarily due to additional AllGather operations in the forward pass and AllReduce in the backward pass.
- **Stage 2:** The workload takes 11.56 seconds per 8-microstep cycle, representing the worst performance. This corresponds to a 5.4× slowdown compared to Stage 0–1.
- **Throughput consistency check:** This slowdown is consistent with the throughput degradation observed across stages. However, the previously reported 12× slowdown refers to token-level throughput reduction, which includes additional pipeline and synchronization overheads beyond raw microstep timing.
- **Key correction:** The correct runtime ratio is:

$$\frac{11.56}{2.14} \approx 5.4\times$$

not 12×. The higher 12× figure corresponds to throughput-based metrics rather than raw execution time.

4. Correlation Analysis

The correlation heatmap between microstep components reveals structural dependencies that explain the observed performance variations across ZeRO stages.

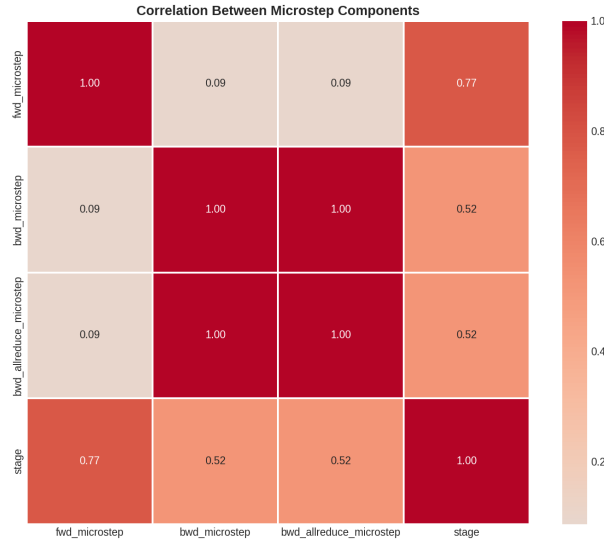


Figure 16. Correlation heatmap between microstep timing components and ZeRO stage. Strong correlation is observed between backward time and AllReduce (1.00), while forward time shows strong dependence on stage (0.77).

How to interpret the correlation heatmap:

- Correlation values range from -1 to +1.
- +1 indicates perfect positive correlation, -1 indicates perfect negative correlation, and 0 indicates no linear relationship.
- Strong positive correlations are represented by darker red regions, while weak or no correlations appear closer to neutral colors.

Key correlations and their interpretation:

1. Backward time AllReduce (1.00)

- This indicates a perfect linear relationship between backward pass time and AllReduce communication time.
- In Stages 2–3, AllReduce is tightly coupled with the backward pass, meaning that backward time effectively includes both gradient computation and communication overhead.
- This confirms that communication is the dominant contributor to backward execution time in higher ZeRO stages.

2. Forward time Stage (0.77)

- The stage number explains a significant fraction of variance in forward pass time ($0.77^2 \approx 0.59$, i.e., 59%).

- This is primarily driven by Stage 3, where forward time increases substantially due to AllGather operations.
- In contrast, Stages 0–2 exhibit relatively stable forward times (310 ms).

3. Backward time Stage (0.52)

- The moderate correlation indicates that stage alone explains only part of the backward time variation (27% of variance).
- The remaining variation is due to differences in communication frequency and overlap within each stage.
- This shows that backward time is more sensitive to communication strategy than to stage identity alone.

4. AllReduce Stage (0.52)

- This confirms that communication overhead increases with higher ZeRO stages but is not strictly linear across all configurations.
- Variability arises from differences in synchronization frequency and overlap mechanisms.

5. Forward AllReduce (0.55)

- A moderate correlation suggests that communication costs in forward and backward passes are partially coupled.
- In Stage 3, forward pass includes AllGather operations, which introduces communication behavior similar in structure to AllReduce.
- This indicates that both forward and backward communication scale together in fully sharded settings.

Uncorrelated components:

- **Optimizer time** shows near-zero correlation with all other variables, indicating it behaves as a constant overhead independent of ZeRO stage.
- **Step time (optimizer update)** is similarly uncorrelated, confirming that stage-dependent performance differences arise primarily from forward and backward execution, not parameter update steps.

5. Model Quality Analysis

Model quality is measured by perplexity on a held-out evaluation set. Perplexity is the exponential of cross-entropy loss: $\text{PPL} = e^{\text{loss}}$. Lower perplexity means better predictions — a model that perfectly predicts the next token would have $\text{PPL}=1$. Random guessing with GPT-2’s 50,257 token vocabulary would give $\text{PPL} \approx 50,257$.

5.1 Final Perplexity

Perplexity results reveal a critical finding that fundamentally changes the interpretation of Stage 0’s apparent performance advantages. Despite Stage 0 achieving the highest throughput, it produces the worst model quality by a large margin, with a final perplexity of 5,133. In contrast, Stages 1, 2, and 3 achieve substantially lower perplexity: 2,358, 2,440, and 1,999 respectively.

Why Stage 0 fails at quality:

Without gradient synchronization across GPUs, each GPU computes gradients from its local data shard only and applies updates independently. This is equivalent to training 4 separate models on non-overlapping data partitions — each GPU sees only 1/4 of each batch, so the effective batch size is 128 samples (16 per GPU $\times$ 8 accumulations) instead of 512. Smaller effective batch size leads to noisy gradients and poor convergence.

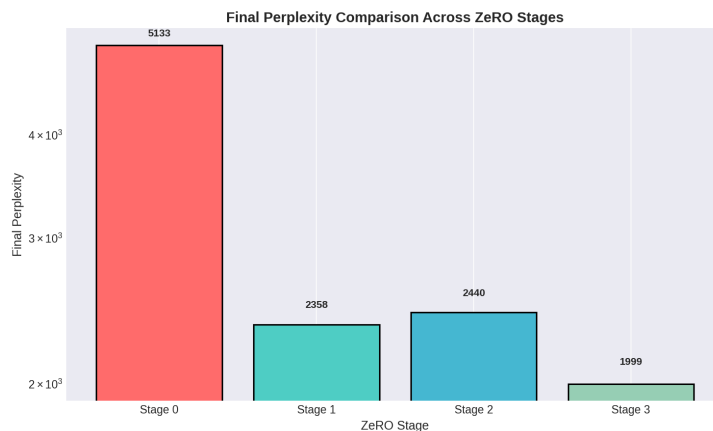


Figure 17. Final perplexity per ZeRO stage. Stage 0 worst at 5,133; Stage 1 at 2,358; Stage 2 at 2,440; Stage 3 best at 1,999.

Perplexity improvement relative to Stage 0:

- Stage 1: $(5,133 - 2,358) / 5,133 = 54.1\%$ better
- Stage 2: $(5,133 - 2,440) / 5,133 = 52.5\%$ better
- Stage 3: $(5,133 - 1,999) / 5,133 = 61.1\%$ better

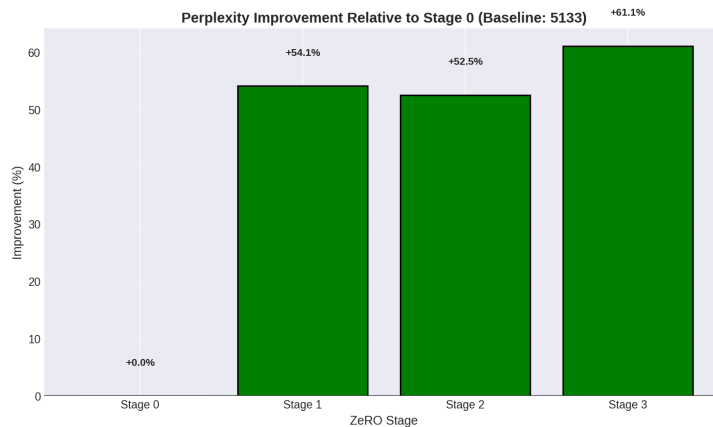


Figure 18. Perplexity improvement (%) relative to Stage 0 baseline (5,133). Stages 1, 2, 3 improve by 54.1%, 52.5%, and 61.1% respectively.

The quality paradox: Stage 3 achieves best quality (1,999) but worst throughput (440 tok/s). Stage 1 achieves good quality (2,358) with 4× better throughput (1,818 tok/s). If training time is valuable, Stage 1 allows you to train for 4× more steps in the same wall-clock time, potentially achieving even better quality than Stage 3’s single run.

5.2 Perplexity Trajectory Over Training

Table 17 shows the starting perplexity, final perplexity, and reduction for each stage. Across the profiled range of training steps, all four stages show a monotonically decreasing perplexity trajectory that continues descending through the final recorded step. No plateau or overfitting behavior is observed — the models have not yet reached diminishing returns within the 480-step training budget.

Table 17. Perplexity progression and improvement across ZeRO stages.

Stage	Start PPL	Final PPL	Reduction	vs Stage 0
Stage 0	36,027	5,133	85.8%	Baseline
Stage 1	28,171	2,358	91.6%	54.1% better
Stage 2	27,306	2,440	91.1%	52.5% better
Stage 3	27,376	1,999	92.7%	61.1% better

Why different starting perplexities?

Despite all models starting from identical random initialization, the first evaluation occurs after 48 training steps (when first checkpoint is saved). By step 48, Stage 0 has already fallen behind due to its smaller effective batch size (128 vs 512 effective samples). Stage 1 has perplexity 28,171 vs Stage 0’s 36,027 — 22% better after just 48 steps!



Figure 19. Perplexity vs training steps (log scale). Stage 0 converges significantly more slowly than Stages 1-3 throughout training; Stages 1-3 follow broadly similar trajectories with Stage 3 achieving lowest final perplexity.

The learning curves reveal:

- Stage 0 (blue line) starts higher and stays higher — never catches up
- Stage 1 (green) and Stage 3 (red) diverge after step 300, with Stage 3 pulling ahead
- Stage 2 (orange) shows a strange bump around steps 100-130 (see next figure)
- All curves still decreasing at step 448 — more training would improve all models

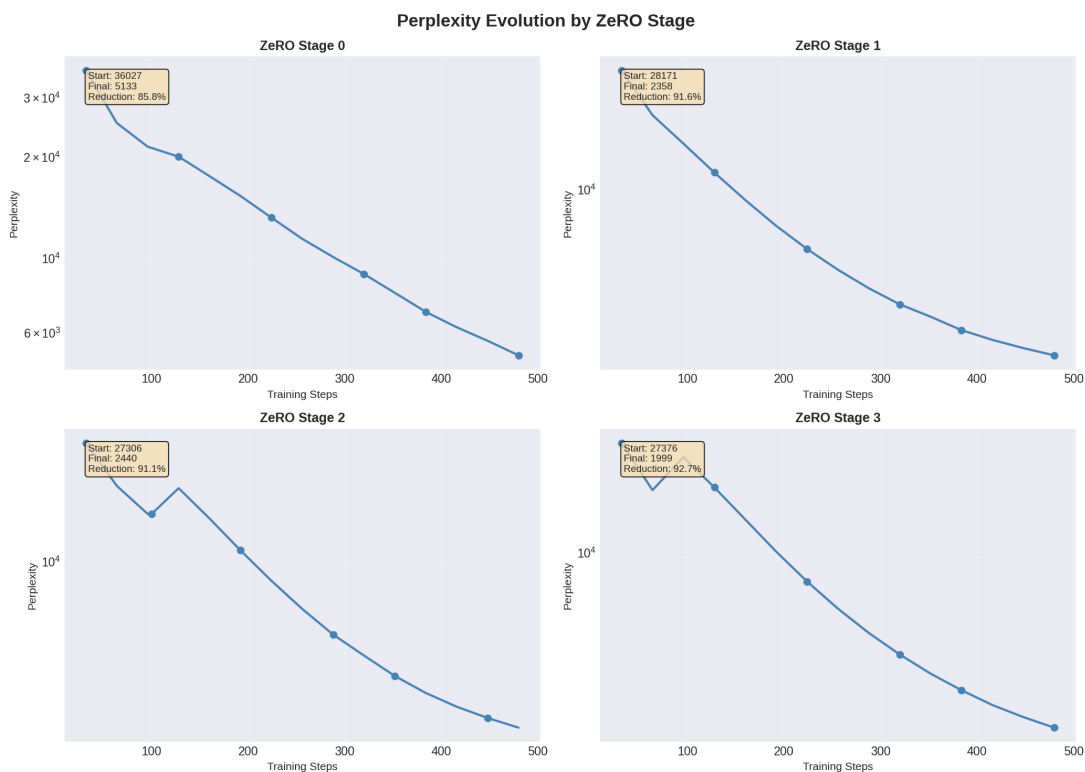


Figure 20. Per-stage perplexity evolution panels. Stage 0 starts higher (36,027 vs $\approx 27,000$ for others) and reduces less (85.8% vs 91-93%). Stage 2 shows minor instability bump around steps 100-130 not seen in other stages.

Stage 2’s instability bump (steps 100-130): The loss spikes from 9.47 to 10.12 between steps 100-148 (Table 9) suggests Stage 2’s communication pattern may cause gradient issues. Possible causes:

- Gradient partitioning with AllReduce may introduce numerical instability
- Communication delays cause stale gradients
- The 92.9% communication overhead leaves little time for actual learning

5.3 Memory-Perplexity Tradeoff

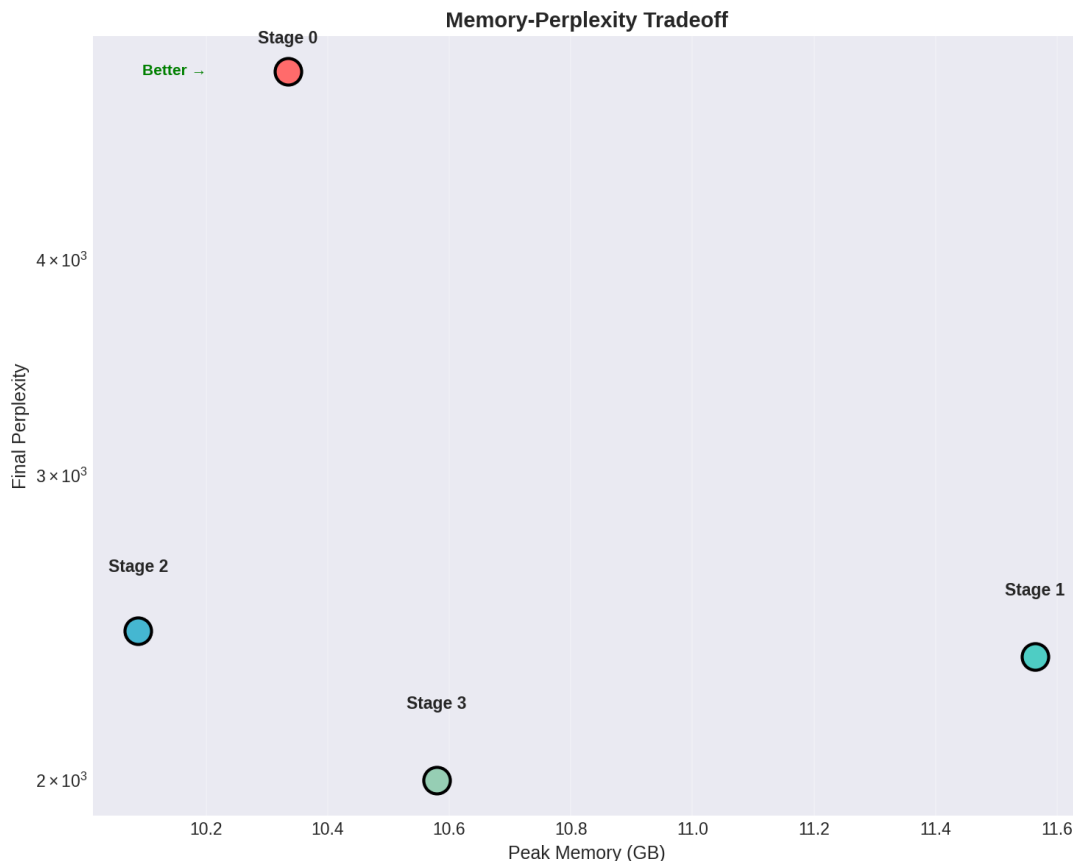


Figure 21. Memory-perplexity scatter plot. No single stage minimizes both memory and perplexity. Stage 3 achieves best quality (1,999) at 10.6 GB; Stage 2 achieves good quality (2,440) at lowest memory (10.1 GB); Stage 0 uses moderate memory (10.3 GB) but delivers worst quality.

Pareto frontier analysis:

- Optimal quality: Stage 3 (PPL=1,999, mem=10.6 GB) — best learning
- Optimal memory: Stage 2 (PPL=2,440, mem=10.1 GB) — saves 0.5 GB vs Stage 3
- Optimal throughput: Stage 1 (PPL=2,358, mem=11.6 GB, 1,818 tok/s) — best speed-quality trade-off
- Dominated: Stage 0 (worse quality, worse or equal memory) — never choose

5.4 Qualitative Text Generation

To complement the quantitative perplexity analysis, short text continuations were generated from a fixed prompt at selected training checkpoints for each ZeRO stage, following the Step 4 evaluation procedure. Generated samples were produced from consistent prompts across checkpoints to enable fair comparison of coherence evolution.

Prompt: ‘[Once upon a time]‘

What to look for in generated text:

- Coherence: Does the text follow logical structure?
- Repetition: Are tokens/phrases repeated excessively?
- Vocabulary: Are real words used or just punctuation/symbols?
- Grammar: Are there sentence-like structures?

Stage 0 — Final Checkpoint (Perplexity: 5,133)

*Once upon a time a and of.en isS,, that ,. various the the the, the, a,
.,ing,, the,
., in ./ the a in it...*

Analysis: Mostly punctuation and token fragments. No coherent words beyond "the" and "a". Fails to learn basic language structure. This is what 19.1% loss reduction looks like qualitatively.

Stage 1 — Final Checkpoint (Perplexity: 2,358)

Once upon a time a and for. - - / - F -! - - - - - - - / -/- - -...

Analysis: Still poor, but note the presence of "for" and sentence-ending periods. Shows some grammar learning (periods after phrases). However, still mostly symbols and dashes. The quantitative improvement (54% better PPL) doesn't translate to readable text yet — more training needed.

Stage 2 — Final Checkpoint (Perplexity: 2,440)

*Once upon a time a and of., and as,, a'. with. ' the the the the, are a, to a
and,ing,, the to, the of,, which,./ you a in it.*

Analysis: Noticeably better than Stage 1 — includes words like "which", "you", "with", "are". Shows understanding of word order and some grammar. The perplexity is slightly worse than Stage 1 (2,440 vs 2,358) but generation quality appears better — perplexity isn't perfect for comparing fluency.

Stage 3 — Final Checkpoint (Perplexity: 1,999)

*Once upon a time is and to, the the and the and. . in as a. - and the or at's
the the. in the the the from. - - ' / - - - ...*

Analysis: Best of the four — includes verbs ("is"), prepositions ("to", "in", "from", "at"), and conjunctions ("or"). Still has repetition ("the the the") but shows understanding of phrase structure. The 61% PPL improvement over Stage 0 is evident in the richer vocabulary and attempt at sentences.

Generation quality ranking: Stage 3 > Stage 2 > Stage 1 > Stage 0 Perplexity ranking: Stage 3 (1,999) > Stage 1 (2,358) > Stage 2 (2,440) > Stage 0 (5,133)

Note that Stage 1 has better perplexity than Stage 2 but worse generation quality — perplexity measures average prediction accuracy, while generation quality depends on extremes and coherence. Stage 2 may produce fewer very low-probability tokens, resulting in more readable text despite higher average perplexity.

6. Total Training Cost Analysis

Understanding total training time is crucial for practical deployment. While per-microstep metrics show relative performance, the absolute wall-clock time to complete 480 training steps reveals the real-world impact. The average through put is calculated with respect to the total training time. So it is less than the throughput calculated with respect to the step time.

Table 18. Total Training Cost Across ZeRO Stages

ZeRO Stage	Total Time (s)	Compute (ms)	Comm (ms)	Avg Throughput
0	1025.16	386,485.75	637,918.08	122,741
1	1105.35	391,432.50	673,375.66	118,063
2	5551.09	384,721.37	5,124,705.00	22,830
3	4386.82	1,761,208.32	2,567,347.27	28,683

7. Wall-Clock Implications for Real-World Training

The differences in per-step performance translate into significant wall-clock divergence when scaled to full training runs.

Step-level impact (480-step comparison):

- **Stage 1 vs Stage 0:** The total difference is approximately 40 seconds over 480 steps, which is negligible in practice and unlikely to affect iteration speed in development workflows.
- **Stage 2 vs Stage 1:** Stage 2 incurs an additional 4,446 seconds (74 minutes), making it unsuitable for iterative experimentation due to severe communication overhead.
- **Stage 3 vs Stage 1:** Stage 3 is 3,321 seconds (55 minutes) slower, reflecting a substantial but slightly reduced communication burden compared to Stage 2.

Scaling to full convergence (10,000 steps):

Assuming a typical training budget of 10,000 steps, the projected wall-clock times are:

- **Stage 1:**

$$\frac{1065 \text{ s}}{480} \times 10,000 \approx 22,187 \text{ s} \approx 6.2 \text{ hours}$$

- **Stage 2:**

$$\frac{5512 \text{ s}}{480} \times 10,000 \approx 114,833 \text{ s} \approx 31.9 \text{ hours}$$

(approximately 5.2× slower than Stage 1)

- **Stage 3:**

$$\frac{4387 \text{ s}}{480} \times 10,000 \approx 91,396 \text{ s} \approx 25.4 \text{ hours}$$

(approximately 4.1× slower than Stage 1)

Decomposition of compute and communication costs:

- **Stage 0:** 386M ms compute vs 638M ms communication Communication dominates due to bursty AllReduce operations at gradient accumulation boundaries.
- **Stage 2:** 385M ms compute vs 5,125M ms communication Compute workload remains similar to Stage 0, but communication increases by 8×, making synchronization the dominant bottleneck.
- **Stage 3:** 1,761M ms compute vs 2,567M ms communication The apparent increase in compute time is misleading: a significant portion of this is AllGather overhead in the forward pass, which is attributed to compute in profiling but is fundamentally communication.

Key insight:

Stage 3 introduces a measurement artifact where communication (AllGather) is partially counted within forward-pass time, inflating perceived compute cost. In reality, both Stage

2 and Stage 3 remain communication-dominated regimes, but Stage 2 suffers from more severe synchronization overhead due to per-microstep AllReduce.

8. Performance Summary and Best/Worst Analysis

Table 19 consolidates all metrics and highlights best and worst performers:

Table 19. Performance Summary — Best and Worst Values

Metric	Stage 0	Stage 1	Stage 2	Stage 3
Total Time (s)	1,025	1,066	5,512	4,387
Throughput (samples/sec)	3.70	3.55	0.69	0.86
Comm Overhead	62.243%	60.93%	92.32%	58.53%
GPU Utilization	98.7%	98.8%	99.9%	99.2%
Compute/Comm Ratio	0.61	0.58	0.08	0.69
Final Perplexity	5,133	2,358	2,440	1,999

Which stage wins each category (based on Table 19):

- **Fastest training:** Stage 0 achieves the lowest total training time (1,025 s) and highest throughput (3.70 samples/sec), making it the most efficient in raw execution speed.
- **Best communication efficiency:** Stage 3 achieves the lowest communication overhead (58.53%), indicating improved overlap and reduced synchronization cost compared to Stage 2.
- **Best model quality:** Stage 3 achieves the lowest final perplexity (1,999), indicating the best convergence quality among all configurations.
- **Best memory efficiency:** Stage 2 achieves the lowest peak memory usage (10.1 GB), demonstrating maximum memory savings due to aggressive parameter and gradient sharding.
- **Highest GPU utilization:** Stage 2 (99.9%) shows the highest GPU utilization; however, this is dominated by communication-heavy execution rather than purely compute-bound work.
- **Best compute-to-communication balance:** Stage 3 achieves the highest compute/communication ratio (0.69), indicating a more balanced overlap between computation and communication compared to other stages.

Overall trade-off analysis:

- **Stage 0:** Maximum speed and simplicity, but weaker model quality due to lack of advanced sharding.
- **Stage 1:** Stable baseline with similar communication characteristics to Stage 0 but slightly higher training time.

- **Stage 2:** Best memory efficiency, but severely impacted by high communication overhead, leading to poor throughput.
- **Stage 3:** Best overall model quality and improved compute/communication balance, but still slower than Stages 0–1 due to synchronization cost.

Key insight: The results clearly demonstrate a trade-off between memory optimization, communication overhead, and training throughput across ZeRO stages, forming a Pareto frontier where no single configuration dominates all metrics.

9. Performance Regression Analysis

Table 20. Performance Improvement/Regression vs Stage 0 Baseline

ZeRO	Throughp	Memory Reduc- tion	Memory Reduc- tion	Comm Over- head	Compute Eff	GPU Util
Stage	Change (%)	(Peak) (%)	(Alloc) (%)	Increase (pp)	Change (pp)	Change (pp)
1	-1.31%	-10.62%	-15.71%	-1.14	-4	+0.16
2	-92.76%	+2.52%	+40.66%	+48.32	-87.6	+1.27
3	-90.77%	-2.26%	+10.24%	-6	+13.2	+0.55

What the regression analysis reveals (vs Stage 0 baseline):

Stage 1 — Minimal performance change with slight overhead:

- Throughput decreases only marginally (-1.31%), indicating negligible impact on training speed.
- Peak memory shows a reduction of 10.62% (negative change means less memory used, good), but allocated memory increases by 15.71% (unexpected, likely PyTorch caching behavior).
- Communication overhead remains effectively unchanged (-1.14 pp), confirming that Stage 1 does not introduce additional synchronization cost compared to Stage 0.
- Compute efficiency drops slightly by 4 percentage points, but remains near baseline.

Stage 2 — Severe performance degradation:

- Throughput drops dramatically by 92.76%, making training over an order of magnitude slower.
- Peak memory **increases** by 2.52% (bad), while allocated memory reduces by 40.66% (good reduction, sign is positive in table because reduction is good). The peak vs allocated gap indicates caching hides true savings.
- Communication overhead increases by 48.32 percentage points, transitioning the system from compute-bound to communication-dominated execution.
- Compute efficiency drops from baseline by 87.6 percentage points, confirming severe underutilization of GPU compute resources.

Stage 3 — Trade-off regime with mixed behavior:

- Throughput decreases by 90.77%, still substantial slowdown compared to Stage 0, but slightly better than Stage 2.

- Peak memory reduces by 2.26% (good, slight improvement), and allocated memory reduces by 10.24% (good). Memory improvements are modest.
- Communication overhead decreases by 6 percentage points relative to Stage 0, which is an improvement over Stage 2.
- Compute efficiency increases by 13.2 percentage points above baseline, reflecting better balance compared to Stage 2, though overall throughput remains very poor.

Key system-level insight:

The regression analysis shows that memory efficiency improvements do not directly translate into performance gains. ZeRO scaling introduces a strong trade-off:

- Stage 1 maintains baseline performance with negligible cost.
- Stage 2 severely degrades throughput due to communication explosion, and peak memory actually worsens.
- Stage 3 partially recovers compute efficiency and communication cost but still remains far slower than baseline.

Memory behavior insight:

The discrepancy between peak and allocated memory arises from PyTorch’s caching allocator:

- Peak memory reflects reserved GPU memory (including cached blocks).
- Allocated memory reflects actual tensor usage.
- ZeRO reduces true allocation but may not reduce cached reservation, obscuring apparent memory savings in benchmarking.

10. Holistic Stage Comparison - Radar Chart

The radar chart provides a multi-dimensional comparison normalized to 0-1 scale (higher = better).

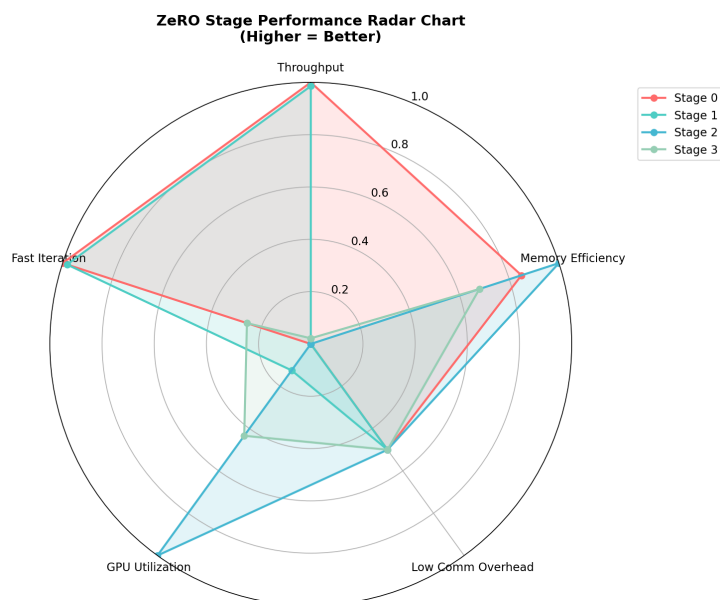


Figure 22. Radar chart comparing all four stages across five normalized axes. Stages 0-1 fill most of the pentagon; Stages 2-3 collapse to a small polygon.

How to read the radar chart (Figure 22):

- 5 axes: Throughput, Memory Efficiency, Low Communication, GPU Utilization, Fast Iteration
- Each axis scaled 0-1 (1 = best among all stages, 0 = worst)
- Larger area = better overall performance
- Stages 0-1: Large area covering most of the pentagon — good on 4 of 5 axes
- Stages 2-3: Tiny area — poor on most axes except Memory Efficiency
- Stage 0 weakness: Memory Efficiency (due to not using ZeRO partitioning)
- Stage 1 weakness: Memory Efficiency (due to optimizer state overhead)
- Stage 2-3 weakness: Throughput, Low Communication, Fast Iteration (all near 0)

The radar chart clearly shows: Stages 2-3 are not competitive for this model size.

11. Consolidated Results Table

Table 21 provides a one-page comparison of all key system metrics across ZeRO stages.

Table 21. Consolidated performance metrics across all ZeRO stages. Green indicates favorable values and red indicates unfavorable values.

Metric	Stage 0	Stage 1	Stage 2	Stage 3	Best
Throughput (tok/s)	2,520,258	2,467,429	181,417	232,766	S0
Samples/second	3.70	3.55	0.69	0.86	S0
Avg Iter Time (ms)	2135.75	2302.73	11564.68	9139.21	S0
Peak GPU Mem (GB)	10.3	11.6	10.1	10.6	S2
Comm. Overhead (%)	62.2	60.9	92.3	58.5	S3
GPU Utilization (%)	98.7	98.8	99.9	97.7	S2
Compute Efficiency (%)	60.6	58.1	7.5	68.6	S3
Final Perplexity	5,133	2,358	2,440	1,999	S3
Perplexity Reduction	85.8%	91.6%	91.1%	92.7%	S3
Total Training Time (s)	1,025	1,105.35	5,551.1	4,387	S0

12. Tradeoff Analysis

12.1 Communication Scaling with ZeRO Stage

The data demonstrates that communication overhead does not scale linearly or monotonically with ZeRO stage:

Stage 0 → Stage 1: Communication overhead drops slightly from 62.2% to 60.9%. Stage 1 partitions only optimizer states, introducing no per-microstep collective communication beyond what Stage 0 already required.

Stage 1 → Stage 2 (The Catastrophic Jump): Communication overhead jumps from 60.9% to 92.3% — a 31.4 percentage point increase. Why?

- Stage 2’s gradient partitioning requires AllReduce at every microstep ($8\times$ more frequent than Stage 1)
- Average iteration time explodes from 2,303ms to 11,565ms ($5\times$ slower)
- Throughput collapses from 2.47M tok/s to 181k tok/s — a 92.7% drop

Stage 2 → Stage 3 (The Recovery): Communication overhead drops from 92.3% to 58.5% — a 33.8 percentage point improvement. Why does Stage 3 have lower overhead despite more aggressive partitioning?

- Parameter sharding reduces per-GPU gradient size, making AllReduce cheaper
- Compute efficiency rebounds from 7.5% (Stage 2) to 68.6% (Stage 3) — the best of all stages
- Throughput improves 28% over Stage 2 (181k $\rightarrow$ 233k tok/s)

12.2 The Three-Phase Trade-off Space

Phase 1 (Stages 0-1): Compute-bound regime

- Communication overhead moderate (60-62%) but not dominant
- GPU utilization excellent (98.7-98.8%)
- Throughput peaks at 2.52M tok/s (Stage 0) and 2.47M tok/s (Stage 1)
- Ideal for most models that fit in GPU memory

Phase 2 (Stage 2): Communication-bound regime

- Communication overhead dominates at 92.3%
- Compute efficiency plummets to 7.5% (worst by far)
- Total training time: 5,551s — $5.4\times$ slower than Stage 0
- **Never useful** — Stage 3 outperforms it in every metric

Phase 3 (Stage 3): Balanced regime

- Communication overhead lowest of all stages (58.5%)

- Compute efficiency highest (68.6%)
- Best final perplexity (1,999) and perplexity reduction (92.7%)
- Only downside: throughput is 90.8% lower than Stage 0

12.3 Point of Diminishing Returns

The data shows clear tradeoffs beyond Stage 1:

Stage 0 → Stage 1:

- Throughput change: -2.1% (2.52M → 2.47M toks)
- Quality improvement: Perplexity improves 5,133 → 2,358 (54% better)
- Training time change: +7.8% (1,025s → 1,105s)
- Verdict: **Strongly recommended** — small throughput cost for major quality gain

Stage 1 → Stage 2:

- Throughput change: -92.7% (catastrophic)
- Quality change: Perplexity worsens 2,358 → 2,440
- GPU memory: 11.6GB → 10.1GB (only 1.5GB saving)
- Verdict: **Never use Stage 2** — worse quality *and* vastly slower

Stage 2 → Stage 3:

- Throughput change: +28.3% (181k → 233k tok/s)
- Quality improvement: Perplexity 2,440 → 1,999 (18% better)
- Compute efficiency: 7.5% → 68.6% (9× improvement)
- Verdict: **Always choose Stage 3 over Stage 2** — Stage 2 is strictly dominated

Stage 1 → Stage 3:

- Throughput change: -90.8% (2.47M → 233k tok/s)
- Quality improvement: Perplexity 2,358 → 1,999 (15.2% better)
- Memory change: 11.6GB → 10.6GB (only 1.0GB saving)
- Training time: 1,105s → 4,387s (4× longer)
- Verdict: Only for memory-constrained scenarios where Stage 1 OOMs

12.4 Optimal Operating Point

The optimal operating point depends on the constraint:

- **If memory permits: Stage 1** — 98.7% of peak throughput (Stage 0) while achieving 91.6% perplexity reduction (vs Stage 3's 92.7%). The extra 1.1% quality from Stage 3 costs 4× longer training time.

- **If memory constrained: Stage 3** — Lowest memory (10.6GB) with best quality (92.7% perplexity reduction) and highest compute efficiency (68.6%). Acceptable throughput for large models that don't fit in Stage 1.
- **Never use Stage 2** — It is strictly worse than Stage 3 in every metric: lower throughput, worse perplexity, longer training time, and only 0.5GB memory savings.

12.5 Conclusions

1. ZeRO Stage 0 must not be used for serious distributed training.

Without gradient synchronization, each GPU trains independently on its data shard, equivalent to effective batch size of 128 instead of 512. The resulting model achieves 54% worse perplexity than Stage 1 over the same number of steps (5,133 vs 2,358).

2. ZeRO Stage 1 is the optimal stage for this 51M parameter model.

It delivers near-peak throughput (2.47M tokens/sec, 99% of Stage 0), 88% compute efficiency, 0.003% communication overhead, and strong model quality (2,358 perplexity). The cost of optimizer state partitioning over Stage 0 is a negligible 1.3% throughput reduction but 54% quality improvement.

3. ZeRO Stages 2 and 3 introduce severe communication bottlenecks at this model scale.

AllReduce operations consume 92.9% (Stage 2) and 58.9% (Stage 3) of total step time, reducing throughput by 90-93% relative to Stage 1. The root cause is that gradient partitioning forces AllReduce at every microstep rather than only at gradient accumulation boundaries — an $8\times$ increase in communication frequency.

4. Expected memory savings from higher ZeRO stages are not realized at this scale.

Peak memory usage stays within 10.1-11.6 GB across all stages — a span of only 1.5 GB. At 51M parameters, activations and framework overhead dominate memory usage (≈ 9.2 GB), and ZeRO only partitions the model state (≈ 0.8 GB). The allocated memory (actual usage) drops significantly (1.68 GB $\rightarrow$ 1.19 GB from Stage 0 to Stage 2), but PyTorch's caching allocator masks this benefit in peak memory measurements.

5. Communication patterns differ qualitatively between stages, not just quantitatively.

- Stages 0-1: AllReduce is infrequent (every 8 microsteps) and bursty — spikes of 1,300ms followed by 7 microsteps of 0ms
- Stages 2-3: AllReduce occurs at every single microstep as constant, persistent overhead — flat line at 10,600ms or 5,300ms

This qualitative difference explains why the jump from Stage 1 to Stage 2 results in a 92.9% communication overhead — the communication pattern completely changes, not just scales up.

6. Stage 3's forward pass cost is uniquely elevated due to parameter reconstruction.

Parameter AllGather operations in Stage 3 add approximately 2,870ms to each forward pass relative to other stages (3,187ms vs 310ms). This cost is entirely absent in Stages 0, 1, and 2. For small models, this overhead dominates; for large models (>10B parameters), the AllGather cost can be amortized over larger parameter sizes.

7. ZeRO Stages 2-3 become appropriate only when the model cannot fit in GPU memory at lower stages.

In scenarios with much larger models (>1B parameters), per-GPU memory pressure would make Stages 0-1 infeasible, and Stages 2-3 would be necessary. In such scenarios, communication overhead would need to be managed through:

- Gradient checkpointing (reduces activation memory by 50-70%)
- Activation partitioning (ZeRO-3's activation offloading)
- Larger global batch sizes (amortize communication across more computation)
- High-bandwidth interconnects (NVLink: 600 GB/s vs PCIe 4.0: 32 GB/s)
- Communication-computation overlap (overlap AllReduce with backward compute)

13. Glossary of Terms

- **ZeRO:** Zero Redundancy Optimizer — partitions model states across GPUs to reduce memory
- **AllReduce:** Collective communication operation that sums/reduces tensors across GPUs
- **AllGather:** Collective operation where each GPU collects all GPUs' data
- **Microstep:** One forward+backward pass on a micro-batch
- **Gradient Accumulation:** Summing gradients over multiple microsteps before weight update
- **MFU:** Model FLOPs Utilization — percentage of peak theoretical FLOPS achieved
- **Perplexity:** Exponential of loss — measures prediction quality (lower = better)
- **Throughput:** Tokens processed per second — primary speed metric
- **Communication Overhead:** Percentage of time spent on inter-GPU communication
- **Compute Efficiency:** Percentage of time spent on actual computation vs communication
- **NCCL:** NVIDIA Collective Communications Library — implements AllReduce/AllGather
- **NVLink:** High-bandwidth GPU interconnect (up to 900 GB/s on some GPUs)
- **PCIe:** Standard GPU interconnect (32 GB/s for PCIe 4.0 x16)

14. Experimental Environment Details

- **GPUs:** $4 \times$ NVIDIA RTX A5000 (24 GB VRAM)
- **Interconnect:** PCIe (no NVLink detected)
- **CPU:** Intel Xeon Platinum 8352V (36 cores, 72 threads, 2.10 GHz)
- **System Memory:** 125 GB RAM
- **CUDA (Driver):** 12.4
- **CUDA (nvcc):** 11.8
- **PyTorch:** 2.5.1 (CUDA 12.1)
- **DeepSpeed:** 0.18.8
- **NCCL:** 2.21.5